UNLOCK

Unlock Underused Equipment and Spaces

Final Theoretical Document and Build Guide

Problem statement, product specification, fairness and safety rules, data model, demo scenario and 48-hour build plan, combined into one reference for building the prototype.

Personal planning document. Contains no links to a live product or repository. All demo data, payments, verification and notifications are simulated.

Contents

1. Executive summary 3
2. Problem statement and evidence 3
3. Product vision and target users 5
4. Personas and empathy maps 5
5. Product concept and transaction model 6
6. Scope decision: one institution, designed for many 6
7. Scope and priorities 6
8. Discovery and resource listings 7
9. Roles, permissions and owner verification 8
10. Eligibility and training verification 8
11. Request, approval and fair allocation workflow 9
12. Availability and booking states 10
13. Scheduling conflicts and competing requests 10
14. Cancellations and no-shows 10
15. Booking extensions 11
16. Collection, return and permanent handover 11
17. Overdue returns 11
18. Damage, incidents and dispute handling 12
19. Transaction history and audit log 12
20. Payment, verification and notifications 12
21. AI use and guardrails 13
22. Dashboards and metrics 14
23. Privacy and security 14
24. Accessibility 14
25. Technical implementation 15
26. Recommended screens 16
27. Seeded demo resources and judge walkthrough 16
28. Acceptance criteria and testing checklist 17
29. 48-hour execution plan 18
30. Repository and documentation 18
31. Master prompt for Google AI Studio Build mode 18
32. Alignment with the problem statement and judging criteria 20
33. Known limitations 20
34. Future development 20
35. Pitch summary 21
36. What to validate next 21
37. Reference platforms for inspiration 21

Document purpose

This is the single combined reference for the UNLOCK prototype. It merges the original problem statement, product specification and build plan with the additional design decisions made afterwards (fair allocation, verification, cancellations, overdue handling, extensions, AI use, privacy and accessibility). It is a guide for building a functional hackathon prototype, not a claim that a production marketplace or real payment system exists.

Two kinds of markers are used. Green boxes marked Decided record choices made by the team. Amber boxes marked Suggested default are proposals that can be changed freely.

1. Executive summary

UNLOCK is a shared-resource discovery and management platform for makerspaces, innovation centres and similar institutions. It helps people discover underused equipment and workspaces and request access under clear rules. Resource owners publish availability and eligibility requirements, review requests, and may earn income from otherwise idle resources. Requesters (students from different colleges, independent inventors, startups and project teams) compare suitable resources, understand the conditions of access, and track a request from submission through collection and return, or permanent handover.

The prototype demonstrates a complete, believable flow using fictional sample data. It prioritises discovery, listing, eligibility checks, fair booking decisions, status tracking and evidence-based incident handling. Payments and verification are simulated and clearly labelled as simulations.

2. Problem statement and evidence

2.1 Challenge

Useful equipment and workspaces may sit idle because potential users do not know they exist, cannot easily compare them, or do not know how to request access. At the same time, people working on projects face high purchase or rental costs, limited access to specialist tools, and unclear booking or training requirements. Ordinary booking systems show availability but rarely address trust, safety and operational requirements involved in sharing institutional property.

2.2 Problem to solve

How might we make underused equipment and spaces easier to discover, access and manage while protecting safety, fairness, resource condition and owner control?

2.3 5W1H problem definition

The problem is defined below using the 5W1H method (Who, What, Where, When, Why, How). The answers are working hypotheses to be validated through interviews.

UNLOCK: Problem Definition using the 5W1H Method

How might we make underused equipment and spaces easier to discover, access and manage while protecting safety, fairness, resource condition and owner control?

Who ?

- **Who are the sufferers?** Students, inventors and startups who cannot afford or find equipment, and owners with idle resources.
- **Who is responsible for the idle resources?** Makerspace owners and lab managers.
- **Who can control access?** Resource owners and assigned supervisors.
- **Who all are involved in it?** Requesters, owners, supervisors and administrators.
- **Who needs immediate help?** Project teams with deadlines and no budget to buy or rent.
- **Who identifies damage or misuse?** Supervisors, using before and after condition records.

What ?

- **What is the issue?** Useful equipment and spaces sit idle while others cannot find, afford or request them.
- **What causes the issue?** Poor discovery, unclear rules, high costs and fear of damage or misuse.
- **What can be done?** A trusted platform to list, discover, request and track access.
- **What steps are necessary?** List, check eligibility, request, review, check out, return, view history.
- **What to be avoided?** Monopolisation, unsafe access and blame without evidence.
- **What may be the reason for failure?** Heavy admin work for owners, or unclear rules for users.

Where ?

- **Where is the problem seen?** Makerspaces, innovation centres, college labs and workshops.
- **Where else can it occur?** Community organisations and small businesses with spare equipment.
- **Where do users look today?** Websites, groups, notice boards and word of mouth.
- **Where to start the solution?** One institution's makerspace, designed to grow to many.
- **Where to concentrate more?** Restricted and high-demand equipment.
- **Where is solution impossible?** Resources that must never be shared through an open platform.

When ?

- **When does the problem occur?** When equipment sits idle between uses while demand elsewhere goes unmet.
- **When can we start clearing the problem?** With a working prototype for one makerspace.
- **When is it more problematic?** Near project deadlines and for popular time slots.
- **When do disputes arise?** At overdue returns, no-shows and damage reports.
- **When to think about a quick solution?** When requests get lost in messages or bookings clash.
- **When is the issue ignorable?** For low-risk items; never for restricted equipment.

Why ?

- **Why does the problem occur?** There is no shared place to discover resources and their rules.
- **Why does it become a problem?** Idle assets waste value, so users pay too much or go without.
- **Why should we find a solution?** Access helps projects and lets owners recover costs.
- **Why should safety and fairness be managed?** Open sharing risks damage, misuse and unfair allocation.
- **Why to solve without creating more issues?** Owners need simple approvals and reliable records.
- **Why can't we ignore it?** Wasted resources, safety risks and no accountability.

How ?

- **How will we find a solution?** One platform for discovery, rules, approval and handover.
- **How to deal with the problem?** Risk-based approval, safety quiz and supervisor review.
- **How to manage the users affected?** Booking caps, a first-eligible waitlist and clear reasons.
- **How to prove condition?** Before and after records with a full transaction timeline.
- **How to solve without creating more issues?** Human review, audit log and simulated payments.
- **How to identify the scale of the problem?** Interviews and utilisation metrics.

Answers are working hypotheses to be validated through interviews with requesters, owners and supervisors.

2.4 Why this matters

- Owners may recover some value from idle resources through permitted temporary access or permanent transfer.
- Requesters may find more affordable options than purchasing equipment outright.
- Clear eligibility, training, availability and condition information reduces uncertainty.
- A shared process makes requests and decisions more traceable than scattered messages or informal arrangements.
- Different resources need different controls: qualifications, training, supervision, deposits or approval.

2.5 Evidence status (hypotheses, not findings)

The team has not yet completed interviews with makerspace owners, equipment supervisors or requesters. Persona statements and empathy maps are research hypotheses. The prototype can demonstrate the workflow but cannot by itself prove that low discovery or low demand is widespread. Never put a statistic in the document unless the source can be named.

Evidence type	What to look for	Status
Secondary research	Published utilisation or access reports from university labs, makerspaces or shared-facility networks	To collect
Existing platforms	How I-STEM and institutional booking pages show availability, training and access rules	To review
Interviews: requesters	3 to 5 people: how they find equipment, what blocks access, how they judge price and trust	To collect

Evidence type	What to look for	Status
Interviews: owners and supervisors	2 to 3 people: listing effort, training, safety, scheduling, damage handling	To collect
Observation	How a facility currently handles bookings (forms, messaging groups, spreadsheets)	To collect

Report sample size and limitations honestly, and update personas and MVP priorities when evidence arrives.

3. Product vision and target users

Vision: one trusted place to discover a resource, understand its rules, request access, and follow the request to completion.

User group	Goals	Main concerns
Requester	Find suitable, affordable equipment or workspace and book it with confidence.	Availability, cost, approval, training, distance, condition, clear instructions.
Resource owner	Make idle resources useful, manage access, recover permitted costs.	Safety, damage, workload, scheduling conflicts, eligibility, accountability.
Equipment supervisor	Confirm training, safe use, handover condition and return status.	Risk controls, records, overdue items, incidents, clear authority.
Platform administrator	Maintain platform rules, resolve escalations, monitor platform health.	Abuse, disputed incidents, permissions, auditability, reliable records.

4. Personas and empathy maps

4.1 Primary persona: requester

A student, independent inventor, startup member or project team member who needs access to a resource but may not own it or know where to find it.

Says	Thinks	Does	Feels
Where can I find the equipment I need? Is there a cheaper option? Can I book this for my project?	Will my request be approved? Will it be available when I need it? Can I trust its condition? Do I meet the training requirements? What if I am rejected?	Searches websites, groups and notice boards. Compares price and availability. Checks eligibility. Submits a request and follows status. Looks for alternatives.	Frustrated when resources are hard to find. Worried about cost and approval. Uncertain about rules. Relieved when an affordable resource is approved.

Core need: easy access to reliable, affordable, well-managed resources with clear rules, safety information and a simple booking process.

4.2 Secondary persona: resource owner

A makerspace or innovation-centre owner, lab manager or authorised facility representative responsible for equipment or workspace that is not used continuously.

Says	Thinks	Does	Feels
How can I let more people use this safely? Who is responsible if it is damaged? I need to control availability. Users must meet training requirements.	Will the requester use it properly? Will approvals create extra work? Can I prove condition before and after? Will a booking interfere with our own work?	Lists equipment and spaces. Sets price, duration, slots and rules. Checks training. Approves, rejects or asks for information. Records handover and return. Reports incidents.	Concerned about safety and liability. Cautious about sharing valuable equipment. Frustrated by no-shows. More confident when rules and records are clear.

Core need: increase useful utilisation of idle resources while keeping control, ensuring safe and eligible use, reducing administrative effort and keeping reliable records.

5. Product concept and transaction model

UNLOCK covers physical equipment (for example, a 3D printer) and workspaces (for example, a project bench or collaboration room). Two decisions are kept separate: whether access is free or paid, and whether the transfer is temporary or permanent.

Price	Transfer	Meaning	Example
Free	Temporary	Borrow	A free one-hour 3D-printer booking
Paid	Temporary	Rent	A two-hour workspace booking with a demo price
Free	Permanent	Giveaway	An owner offers an unused item to keep
Paid	Permanent	Sale	An owner lists an item for sale

Temporary access needs a booking window and, where relevant, collection and return. Permanent transfer needs its own handover workflow and is never treated as a normal time-slot booking.

6. Scope decision: one institution, designed for many

Decided. The prototype serves one institution (for example, one college makerspace network), but the data model is built so more organisations can be added later.

- Every record (resource, request, booking, incident, audit log) carries an organisationId and a facilityId.
- Rules such as caps, cancellation policy and grace periods are stored per organisation and per resource, never hard-coded.
- Roles are scoped: an owner or supervisor acts only inside their own organisation and facility.
- Requesters from other colleges are supported, but they get no write access to another organisation's data.
- This keeps the demo small and reliable while showing a realistic path to multi-organisation adoption.

7. Scope and priorities

7.1 MVP: build these first

- Homepage with resource search, categories and filters.
- Resource listing and detail pages for equipment and workspaces.
- Clear price and transfer fields, availability, eligibility, training, safety notes and owner rules.
- Requester flow: check details, submit request, see status, receive in-app status updates.
- Owner flow: create and edit listings, view requests, approve, reject or ask for more information, manage availability.
- Risk-based approval: only eligible low-risk requests may auto-confirm when the owner has enabled it; restricted resources require review.
- Safety checklist or quiz plus supervisor approval for restricted resources.
- Conflict-safe availability: prevent overlapping confirmed bookings on the backend.
- Fair allocation: per-user booking cap, waitlist with first-eligible order, and decision explanations with reasons.
- Cancellation, extension and overdue rules as owner-defined policies.
- Collection and return records for temporary access, and a distinct handover flow for permanent transfers.
- Incident reporting with human review, and an audit log of important actions.
- Basic role-specific dashboards and a small set of metrics.
- Seeded fictional data and a repeatable demo path.

7.2 If time remains

- Email notifications in addition to in-app notifications, only if a real email integration is configured.
- Evidence photo attachments for handover and return condition.
- AI resource matching and AI incident summaries (with rule-based fallback).
- Simple utilisation charts and demand indicators.
- Saved or favourite resources and search ranking improvements.

Note: the waitlist and booking cap moved from the optional list into the MVP, because fair allocation is part of the core problem. If time runs short, cut AI features and charts before any trust or fairness feature.

7.3 Explicit prototype limitations

- Payments are simulated; do not collect real card or bank details.
- Verification is simulated unless a real, approved integration is implemented.
- Email is not considered sent unless a configured service actually sends it.
- Do not claim real partner facilities, actual users or validated demand without evidence.

8. Discovery and resource listings

8.1 Homepage

- Prioritise discovery before sign-in: search bar, categories, filters, resource cards.
- Show sign-in entry points and role dashboards after authentication.
- Mobile-friendly responsive layouts and accessible labels.

8.2 Search and filters

- Keyword search across name, description, category and supported use.
- Category: equipment or workspace, plus more specific categories.
- Availability: available now, upcoming slots, or unavailable.
- Price: free or paid. Transfer: temporary or permanent.
- Eligibility and training requirements, location and facility.

8.3 Listing fields

Field group	Suggested fields
Core	Title, resource type, category, description, photos, owner and facility, location, status.
Access model	Free or paid; temporary or permanent; price and unit if paid; booking duration or transfer terms.
Availability	Simple status or scheduled slots; blackout periods; booking lead time if needed.
Eligibility and safety	Risk level, training required, allowed users, age or authorisation restrictions, supervision, PPE and safety instructions, prohibited uses.
Condition and logistics	Condition summary, included accessories, collection point, setup notes, return instructions.
Owner controls and policies	Approval mode, minimum notice, booking cap, cancellation notice, no-show rule, simulated fee, grace period, extension rule, published priority criteria, owner contact route.

Use mandatory core fields for every listing. Add mandatory safety and eligibility fields according to the resource's risk. Allow optional custom details, but do not let custom fields replace required safety controls.

9. Roles, permissions and owner verification

9.1 Roles

Role	Allowed actions	Restrictions
Requester	Search, view public listings, request access, take quizzes, track own requests, cancel, request extensions, submit condition feedback or incident reports.	Cannot approve own request, edit another owner's listing, or view restricted evidence.
Resource owner	Create and manage own listings, set access rules and policies, review requests, record handover and return, report incidents.	Cannot override platform permissions or access unrelated private records.
Equipment supervisor	Verify training, approve high-risk eligibility, supervise restricted access, record condition and return, flag incidents.	Only acts on assigned facilities and resources.
Platform administrator	Manage accounts and roles, spot-check owners, handle escalations, audit decisions, moderate listings.	Sensitive actions must be logged and limited to authorised admins.

Enforce permissions in backend rules or server-side checks, not only by hiding buttons. A user must not be able to change their own role or access another person's request by editing a URL or client state.

9.2 Owner and facility verification

Decided. Owners self-register and an administrator spot-checks.

Owner state	Meaning
Self-registered	Can create draft listings; listings carry an 'Unverified owner' label.
Spot-checked	Admin has reviewed a sample of listings and owner details (label: 'Demo verified').
Flagged	Admin concern raised; new listings paused pending review.
Suspended	Listings hidden; existing bookings handled by admin.

Admin spot-check checklist: the owner is linked to a facility or institution; listing details match the rules for the risk level; safety fields are filled; the condition description is plausible; a contact route exists.

Suggested default. High-risk listings are always checked by an admin before they become bookable, even for self-registered owners. All verification labels are simulated in the prototype.

10. Eligibility and training verification

Decided. A short safety checklist or quiz per resource, with supervisor approval for restricted or high-risk equipment.

Risk level	Check required	Who confirms
Low (hand tool, meeting table)	Account check and acknowledging the rules	Automatic, per the owner's policy
Medium (3D printer)	Safety quiz or checklist	System scores it; owner policy decides whether review is needed
High (laser cutter, heavy machine)	Quiz plus supervisor approval	Authorised supervisor must approve

- A user without the required completed training never receives access; the booking stays pending or is rejected with a reason.
- Store per user and per resource: quiz result, attempt count, verification status, approving supervisor, approval timestamp.
- Quiz content is written by the owner or supervisor, because only they know the risks of their equipment.

- Clearly label everything 'Demo verified' or 'Simulated verification'. Never imply that a real identity or qualification was verified.

Suggested default. Approvals expire after about 12 months or when safety requirements change. Limit quiz attempts (for example 3 per day) so the quiz cannot be brute-forced.

11. Request, approval and fair allocation workflow

11.1 Workflow

1 Requester opens a listing and reviews price, transfer type, availability, condition, eligibility, safety requirements and published policies (cap, waitlist, cancellation).

2 The system checks that the listing is active, the slot is valid, and the requester satisfies required eligibility where it can be checked.

3 Requester submits a request. The system stores a request ID, timestamp, requested slot or transfer, and status.

4 For low-risk resources the request may auto-confirm only if the owner has explicitly enabled it and all checks pass.

5 For restricted or higher-risk resources the request stays pending until the authorised owner or supervisor reviews it and confirms training or supervision.

6 The owner may approve, reject with a reason, or ask for more information. The system records actor, time and decision.

7 A confirmed temporary booking reserves its slot. The backend rejects overlapping confirmed bookings.

8 If unavailable, the requester may pick an alternative or join the waitlist.

9 The requester tracks status and receives in-app notifications when it changes.

11.2 Fair allocation: caps and waitlist

Decided. A cap on active bookings per user for high-demand resources, plus a transparent first-eligible waitlist. Users must meet all safety and training requirements before being considered eligible. Allocation decisions are recorded, and users cannot bypass eligibility or manipulate the queue.

Order of decisions: (1) safety and eligibility are prerequisites, never ranking factors; (2) check the user's cap for that resource; (3) if a slot is free, the request proceeds to approval; (4) if not, the user joins the waitlist.

Cap rules

- The owner sets the cap per resource according to demand and policy. Suggested default: 2 active (future or in-use) bookings per user for resources marked high-demand.
- The cap and the waitlist rule are published on the listing before anyone requests.
- Rules are not changed after seeing who has requested. Changes apply only to future requests and are logged.

Waitlist rules

- Order is by the server timestamp at which the user became eligible and joined. The tie-breaker is the request ID, which is documented.
- When a slot opens, it is offered to the first eligible person. Eligibility is re-checked at the moment of the offer.
- Suggested default: offers last 24 hours (or a shorter owner-set window); if not accepted, the offer passes to the next person without harming the first user's standing.
- Anti-manipulation: one waitlist entry per user per resource; spots cannot be transferred or sold; a user at their cap cannot join; leaving and rejoining puts the user at the back.
- Every allocation decision is logged with the rule that was applied.

11.3 Decision explanations

Every approval, rejection and waitlist placement shows the user a plain-language reason, using fixed reason codes plus an optional owner note. Each explanation links to the published rule that was applied and appears in the audit log.

Outcome	Example reason shown to the user
Approved	Eligible and slot free; approved by [role] on [date].
Rejected	Training incomplete; slot unavailable; cap reached; resource under maintenance; owner declined (with note).
Waitlisted	Slot taken; you are number N; the first eligible person is offered the slot; here is the published rule.
More information needed	Specific question from the owner and a deadline to answer.

For the MVP, a transparent first-eligible rule with a cap is easier to explain than a complex scoring algorithm.

12. Availability and booking states

Availability can be a simple status for basic listings or scheduled slots for bookable resources. If slots are used, validate them on the backend at request time and at confirmation time.

State	Meaning
Available	Resource or slot can accept requests.
Pending review	A request exists but is not yet confirmed.
Waitlisted	No slot available; the user holds an ordered waitlist position.
Offered	A freed slot has been offered to a waitlisted user for a limited time.
Confirmed	Authorised booking accepted and slot reserved.
Rejected	Owner or supervisor declined; reason shown where appropriate.
Cancelled	Request or booking cancelled under the published rules.
In use / checked out	Temporary resource has been handed to the requester.
Returned / completed	Return or temporary booking is complete.
Overdue	Expected return time passed and the item has not been recorded as returned.
Unavailable / maintenance	Owner has blocked access, for example for repair or internal use.

13. Scheduling conflicts and competing requests

- Overlap prevention: the backend rejects a confirmation that overlaps an existing confirmed booking. The check and the write happen as one transaction, so two simultaneous confirmations cannot both succeed.
- Competing pending requests for one slot: the first eligible request is reviewed first. Others stay pending or move to the waitlist with an explanation. Approving one request never silently rejects another.
- Owner blocks a slot after a booking is confirmed (maintenance or internal use): the affected user is notified, offered alternative slots or resources, and given priority on the next matching slot. The block is logged with a reason.
- Suggested default: pending requests expire if not reviewed within 48 hours, and the requester is told.

14. Cancellations and no-shows

Decided. Owner-defined cancellation and no-show policy per listing, shown before the request on the resource page and again on the booking confirmation page.

Fields the owner sets: cancellation notice period; simulated cancellation fee (if any); what counts as a no-show (for example 15 minutes after slot start); consequence of repeated no-shows.

1. User cancels; the system checks the notice period and records the cancellation as timely or late.
2. The slot returns to availability and the first eligible waitlist user is offered it.
3. The owner and the waitlisted user are notified.
4. No-shows are recorded by the owner or supervisor.
5. Repeated no-shows are flagged to the owner for review. Nothing is penalised automatically.
6. Emergencies and disputed cases are reviewed by a person, and the user can add an explanation.
7. Fees are simulated and labelled as such.

15. Booking extensions

Suggested default. Extension rules proposed for the prototype.

- The requester asks before the booking ends. The system checks that the next slot is free, nobody is on the waitlist for it, and the user is in good standing (no overdue items).
- Low-risk resource with owner-enabled auto-extension: confirmed automatically, up to one extension of at most the original length.
- Any other case: the owner or supervisor approves or rejects with a reason.
- If a waitlist exists, the extension is refused and the user is told why. This protects fairness.
- Extensions are logged; for paid resources the additional simulated charge is recorded separately.
- A booking that is already overdue cannot be extended.

16. Collection, return and permanent handover

16.1 Temporary access

- At collection, record timestamp, responsible staff member, visible condition, accessories and optional photos.
- At return, record timestamp, condition, accessories returned and any reported issue.
- If the return is late, mark the booking overdue and notify the relevant parties.
- A damage report includes evidence and a chance for relevant parties to respond.

16.2 Permanent transfer

- Keep sale and giveaway separate from temporary booking states.
- Record transfer terms, recipient, owner confirmation and handover date.
- For paid transfers, record only the simulated payment state.
- Mark the transfer complete only after the required handover steps are recorded.

17. Overdue returns

Decided. Staged workflow with human review and no automatic financial penalties in the prototype.

Stage	What happens
1. Reminder	Reminder to the requester shortly before the due time and at the due time.
2. Overdue flag	If not returned, the booking is marked Overdue; the requester and owner are notified.
3. Grace period	Configurable by the owner; the user can still return or report an emergency.

Stage	What happens
4. Temporary restriction	After the grace period the user cannot make new temporary bookings until the item is returned or an owner or supervisor reviews the case.
5. Resolution	Return recorded, restriction lifted, outcome logged.

Users can report emergencies or dispute the return record. Authorised staff review exceptions. Every status change and decision is recorded. Any penalty shown is simulated and labelled.

18. Damage, incidents and dispute handling

The team proposed that a user who damages equipment should pay for the damage. UNLOCK supports accountability but must not automatically assign blame merely because someone was the last user.

- Compare documented condition before and after access.
- Consider photos, timestamps, supervisor notes, equipment logs, normal wear, pre-existing damage and user responses.
- Route routine incidents to the owner or supervisor; escalate serious or contested cases to an administrator.
- Keep an incident report as an allegation until reviewed; restrict evidence access to authorised people.
- Record the outcome, rationale and any agreed resolution.
- In a real deployment, any repair charge or compensation needs clear published terms and a fair review process. In the prototype, use a simulated resolution only.

Repeated-problem flags (rule-based, not AI)

- Counters per user: late returns, no-shows, late cancellations, incident reports naming the user.
- When a counter passes an owner-set threshold, the owner or supervisor sees a flag in their queue.
- A flag prompts review. It never restricts a user by itself, and the user can respond.

19. Transaction history and audit log

Every request has one timeline visible to the requester and the owner (and the supervisor or admin where relevant). Every important action is also written to an append-only audit log.

Events recorded: request submitted; eligibility check result; decision with reason; waitlist changes; payment state changes (simulated); extension requests; check-out with condition; reminders; return with condition; overdue flags; incident reports and responses; final outcome.

Visibility: the basic timeline is shared by both parties. Evidence (photos, incident details) is visible only to authorised roles. Each entry shows actor, role, time and reason.

20. Payment, verification and notifications

20.1 Payment simulation

Do not integrate real payments. Keep payment status separate from booking or transfer status.

Payment status	Interpretation in demo
Not required	Free access or no payment applies.
Pending	Payment is expected but has not been recorded.
Marked as paid	Requester claims payment was made; not independently verified.
Cancelled	Simulated payment was cancelled.
Refunded (simulation)	A refund is recorded in the demo only; no real money moved.

20.2 Verification

- Label any demo verification badge as 'Demo verified' or 'Simulated verification'.
- Use risk-based checks: low-risk items need basic account checks; restricted resources need training evidence, owner or supervisor approval, or supervised access.
- Never imply the platform verified a real identity or qualification unless an actual approved process did so.

20.3 Notifications

Event	Recipient
Request received	Owner or supervisor
Approved, rejected, more information needed	Requester
Waitlist position change and slot offer	Requester
Booking reminder and return reminder	Requester
Cancellation	Owner and affected waitlist user
Overdue flag and restriction	Requester and owner
Return recorded	Owner and requester
Incident opened or updated	Relevant parties

In-app notifications come first. Add email only if a real service is configured. Mocked notifications are labelled as demo notifications.

21. AI use and guardrails

Decided. AI is used for two purposes only: resource matching and incident-report summarisation. A human always makes the final decision.

1 Resource matching: recommends equipment and workspaces from the requester's project needs, budget, availability, location and eligibility.

2 Incident summarisation: converts submitted reports into a neutral summary of the reported issue, timeline and available evidence.

Guardrails

- AI recommendations never override eligibility rules, safety requirements, published allocation policy or availability. They only rank options that already pass those checks.
- A human owner, supervisor or authorised administrator makes all final decisions on restricted access, incident findings, responsibility for damage and penalties.
- Summaries separate reported allegations from verified facts, must not invent missing details, and are editable by the reviewer.
- All AI output is labelled as AI-generated.
- Fallback: if AI is unavailable, matching uses keyword search and filters, and incident reports are shown in full.
- Suggested default: send the AI only the minimum text it needs, without names, emails or contact details.

22. Dashboards and metrics

Dashboard	Useful content
Requester	My requests, status, upcoming bookings, waitlist positions, overdue items, alternatives, notifications.
Owner	My listings, availability, pending requests, confirmed bookings, blocked slots, overdue returns, incidents, repeated-problem flags.
Supervisor	Assigned resources, training checks and approvals, upcoming handovers, returns due, condition records, incident queue.
Admin	User and role overview, owner spot-checks, escalations, audit log, resource status, basic platform metrics.

MVP metrics

- Number of listed resources and number currently available.
- Requests submitted, confirmed, rejected and pending; waitlist count.
- Overdue returns and open incidents.
- Utilisation, with a clear denominator: confirmed booked hours divided by published available hours for scheduled resources.
- Do not present seeded demo counts as real-world impact. Label them as sample data.

23. Privacy and security

Data	Purpose	Who can see it
Name, email, role	Account and notifications	The user; admin; owner or supervisor for their own requests
Quiz result and approval status	Eligibility	The user; supervisors; owner
Request and booking records	Operations	The two parties; supervisor; admin
Condition photos and incident evidence	Accountability	Authorised roles only
Audit logs	Review of decisions	Admin; parties see their own entries

- Data minimisation: collect only what the workflow needs. No real ID numbers, payment details or sensitive documents.
- Permissions are enforced on the server, not only by hiding buttons.
- Role assignment is controlled by admins only; users cannot change their own role.
- Evidence files use access rules tied to the request and role.
- Audit logs are append-only, and sensitive admin actions are logged.
- Suggested default: evidence and incident records are kept only as long as needed to resolve the case, then deleted or anonymised, with the policy stated in the app.
- Demo data is fictional. No real payments, verification or facility data is used.
- Do not claim production-grade security.

24. Accessibility

- Full keyboard navigation and visible focus states.
- Sufficient colour contrast; status is never shown by colour alone (chips include text and an icon).
- Labels on all form fields and clear error messages.
- Plain language for rules, with short explanations of borrow, rent, giveaway and sale.

- Mobile-first layouts with large tap targets.
- Meaningful alt text on listing photos.
- Useful empty, loading and error states.

25. Technical implementation

Choose the stack the team can implement and debug fastest. A practical option is React with TypeScript for the frontend and Firebase (Authentication, Firestore, Storage). A separate backend or API is also valid. Do not add technologies just to make the architecture look complex.

Layer	Responsibilities
Frontend	Responsive discovery pages, listing details, request forms, quizzes, dashboards, status timelines, accessible UI.
Authentication	Sign-in and identity; role assignment controlled securely.
Database and backend	Listings, slots, requests, decisions, waitlists, condition logs, payment simulation, notifications, incidents, audit records.
File storage	Listing photos and authorised condition evidence with access rules.
Server-side rules	Role checks, eligibility, state transitions, overlap prevention, cap and waitlist logic, audit logging, validation.
AI layer (optional)	Matching and incident summaries through an API, with rule-based fallback and minimal data sent.

25.1 Data flow in one line

Frontend action, then server-side validation (role, eligibility, cap, overlap), then database write in a transaction, then audit log entry and notification. Nothing important is decided only on the client.

25.2 Data model

Entity	Key fields
organisations	id, name, settings (default caps, grace periods)
users	id, displayName, email, role, organisationId, verificationState, userStanding, createdAt
ownerVerification	ownerId, state, checkedBy, checkedAt
resources	id, ownerId, facilityId, organisationId, title, type, category, description, photos, location, priceMode, transferMode, price, riskLevel, eligibility, trainingRequired, safetyNotes, approvalMode, policies, availabilitySlots, status
policies (per resource)	bookingCap, cancellationNotice, noShowRule, simulatedFee, graceMinutes, extensionRule
eligibilityRecords	id, userId, resourceId, quizResult, attempts, status, approvedBy, approvedAt, expiresAt
requests	id, resourceId, requesterId, requestType, slotId or transferTerms, status, eligibilitySnapshot, message, createdAt, decisionBy, decisionAt, decisionReason
waitlistEntries	id, resourceId, userId, joinedAt, status, offerExpiresAt
decisionReasons	code, text, ruleReference
handoverRecords	id, requestId, checkoutAt, checkoutBy, beforeCondition, beforePhotos, dueAt, returnedAt, returnCondition, returnPhotos
paymentsDemo	id, requestId, amount, currency, status, markedBy, updatedAt, simulationLabel
incidents	id, resourceId, requestId, reportedBy, description, evidenceRefs, status, response, decision, decisionReason, resolvedAt
userStanding	lateReturns, noShows, lateCancellations, restrictedUntil
notifications	id, recipientId, type, message, relatedRecordId, readAt, createdAt
aiOutputs	id, type, relatedId, text, generatedAt, editedBy
auditLogs	id, actorId, action, entityType, entityId, timestamp, summary

Use server timestamps. Validate all state transitions server-side. A client-side check alone cannot prevent duplicate bookings or unauthorised actions.

26. Recommended screens

1. Landing and discovery page: search, categories, filters, featured resources.
2. Resource details page: photos, description, price and transfer, availability, safety, eligibility, owner policies.
3. Sign-in and account screen with role-aware navigation.
4. Safety checklist or quiz screen.
5. Requester dashboard: requests, status, waitlist, upcoming booking, notifications.
6. Owner dashboard: listings, availability controls, request review queue.
7. Create and edit listing form with conditional safety fields and policy settings.
8. Request details and review page with approve, reject and request-more-information actions.
9. Handover and return condition form; permanent transfer handover page.
10. Incident report and review screen.
11. Transaction timeline view.
12. Admin overview, owner spot-check and audit page.

27. Seeded demo resources and judge walkthrough

Use fictional demo data and label it clearly. The purpose is to demonstrate workflow, not to imply a real partnership.

Demo resource	Access model	Key rule	Demonstrates
3D printer	Free + temporary (borrow)	Safety quiz; pending until supervisor review; booking cap and waitlist	Discovery, eligibility, request, review, fair allocation
Project workspace	Paid + temporary (rent)	Fictional hourly price; simulated payment only	Filters, booking, separate payment state, cancellation policy
Unused hand tool	Free + permanent (giveaway)	Owner confirms recipient and handover	Permanent transfer flow distinct from booking
Surplus equipment	Paid + permanent (sale)	Demo price and simulated payment	Sale listing and handover completion

Suggested 3-minute demo

1. Start on the discovery page and filter for a free, temporary resource.
2. Open the 3D-printer listing and show condition, availability, safety notes, training requirement and published cap and waitlist rule.
3. As a requester, complete the safety quiz and submit a request; show it is pending, not falsely confirmed.
4. Switch to the supervisor view, verify training, approve or reject with a reason.
5. Return to the requester dashboard and show the updated status, explanation and notification.
6. Show a second requester placed on the waitlist with a clear explanation and position.
7. Show the workspace booking with a fictional price and clearly simulated payment state.
8. Record check-out and return condition, then open the transaction timeline.
9. Show an incident report and explain how it is reviewed fairly.

28. Acceptance criteria and testing checklist

- A visitor can search and filter resources without signing in, subject to the public-information policy.
- A listing clearly shows equipment or workspace, free or paid, temporary or permanent.
- Required safety and eligibility fields appear when the resource risk requires them.
- A requester can submit a valid request and see its status.
- A pending request is never displayed as a confirmed booking.
- A restricted resource cannot be auto-confirmed without the required authorised review.
- A user without completed training cannot receive access.
- The backend prevents overlapping confirmed bookings for the same resource.
- The booking cap is enforced, and the waitlist offers freed slots in first-eligible order with a logged reason.
- Every approval, rejection and waitlist outcome shows a plain-language reason.
- A user cannot approve their own request, and requester accounts cannot grant themselves owner privileges.
- Owners can manage only their own resources and assigned facilities.
- Cancellation, extension and overdue rules follow the owner's published policy.
- An overdue booking triggers reminder, flag, and only after review a temporary restriction.
- Permanent transfers use a handover status, not temporary return states.
- Payment simulation status does not imply booking completion or verified payment.
- Condition records and incident evidence are visible only to authorised roles.
- An incident does not automatically assign liability without review.
- AI output is labelled, editable, never decides access or penalties, and has a fallback.

- Each transaction has a complete timeline viewable by both parties.
- Demo verification, payments, data and notifications are labelled as simulated.
- The main flow works on a mobile viewport with useful error and empty states.

29. 48-hour execution plan

Hours	Focus	Deliverable
0-3	Align	MVP confirmed, roles assigned, repo created, data model and visual direction agreed.
3-8	Foundation	App shell, routing, sample data, authentication and role approach, discovery page.
8-16	Listings	Resource cards, detail page, filters, create and edit listing with risk-based fields and policies.
16-25	Core workflow	Request submission, eligibility check, owner review, statuses, availability checks, overlap prevention.
25-32	Operations	Requester and owner dashboards, handover and return records, notifications.
32-37	Trust controls	Safety quiz and supervisor approval, simulated payment, incident flow, permissions, audit log.
37-42	Fairness and lifecycle	Booking cap, waitlist, cancellation and extension rules, overdue stages, decision explanations, transaction timeline.
42-46	Demo preparation	Seed clean data, rehearse the judge flow, QA against the checklist, screenshots and fallback recording.
46-48	Finish	Final README, setup instructions, known limitations, pitch and final build check.

Suggested team ownership

- Frontend lead: discovery, listing details, forms, dashboards, responsive UI.
- Backend and data lead: schema, permissions, state transitions, booking conflict checks, waitlist logic.
- Workflow and UX lead: user journeys, copy, safety and fairness rules, decision explanations.
- Integration and demo lead: seed data, QA checklist, deployment, README, pitch and rehearsal.

If the team is smaller, combine roles, but keep one person responsible for end-to-end integration and final testing.

30. Repository and documentation

Suggested structure: unlock/ with README.md; docs/ (problem-and-scope, user-flows, data-model, testing-checklist); src/ with components, pages, services, types and features folders for resources, requests, availability, waitlist, eligibility, handover, incidents and notifications; public/demo-assets; tests/.

The README should include the problem statement, key features, tech stack, setup steps, demo scenario, screenshots, limitations, and which integrations are simulated.

31. Master prompt for Google AI Studio Build mode

Paste this prompt as a starting point. Review generated code, test all flows, and iterate with smaller follow-up prompts rather than assuming the first generation is correct. This version includes the extra decisions made after the original prompt.

```
Build a responsive web application prototype named UNLOCK for the problem statement "Unlock Underused Equipment and Spaces."

GOAL: Help students, independent inventors, startups and project teams discover and request access to underused makerspace and

innovation-centre equipment and workspaces. Help owners publish resources, control availability and eligibility, review requests,
```

```
and record handover and return. Single institution for now; every record carries organisationId and
facilityId for future growth.
DESIGN: Clean, modern, accessible UI, dark navy and white base with restrained violet and blue accents.
Discovery is the homepage
priority. Mobile layouts, clear status chips (text plus icon), helpful empty and error states. No fake
partner logos.
ROLES: requester, resource owner, equipment supervisor, platform administrator. Enforce roles
server-side where possible.
TRANSACTION MODEL: price (free or paid) and transfer (temporary or permanent) are independent.
Free+temporary = borrow,
paid+temporary = rent, free+permanent = giveaway, paid+permanent = sale. Temporary uses booking slots
and return records;
permanent uses a separate handover workflow.
PAGES: discovery home; resource detail; sign-in; safety quiz; requester dashboard; owner dashboard;
create/edit resource with
conditional safety fields and policy settings; request review page; collection/return form and
permanent handover page;
incident report and review; transaction timeline; admin and audit view.
RULES:
- Requests are pending unless all checks pass AND the resource is low-risk AND the owner enabled
auto-confirm.
- Medium and high-risk resources need a safety quiz; high-risk also needs supervisor approval. No
training, no access.
- Prevent overlapping confirmed bookings with a transactional server-side check.
- Fair allocation: eligibility first, then a per-user cap on active bookings for high-demand resources
(default 2), then a
waitlist ordered by first eligible request with request ID as tie-breaker. Offers last 24 hours,
eligibility is re-checked, one
waitlist entry per user, spots are not transferable. Log every allocation decision.
- Show a plain-language reason for every approval, rejection and waitlist placement.
- Owner-defined cancellation notice, no-show rule and simulated fee, shown on the listing and
confirmation page.
- Extensions: allowed only if the next slot is free, nobody is waitlisted, and the user is in good
standing; auto-extend only for
low-risk resources where the owner enabled it; otherwise owner approval.
- Overdue: reminder, then overdue flag, then after a grace period a temporary restriction on new
temporary bookings pending
human review. No automatic financial penalties. Allow emergency and dispute reports.
- Flag repeated late returns and no-shows for owner review; never auto-penalise.
- Payment state is separate from booking state: Not required, Pending, Marked as paid (unverified),
Cancelled, Refunded (simulation).
- Record condition before and after use. An incident is an allegation until reviewed; no automatic
blame; restrict evidence to
authorised roles. Append-only audit log of all important actions.
- Owners self-register; admin spot-checks. Label owners Unverified or Demo verified.
- AI only for (1) resource matching and (2) neutral incident summaries that separate allegations from
verified facts. Label AI
output, keep it editable, never let it override rules or decide access, penalties or damage
responsibility, and provide a
keyword-search fallback.
- Label all mock payments, verification, seeded data and notifications as simulated.
DEMO DATA: 3D printer (free, temporary, one-hour slots, quiz plus supervisor review); project workspace
(paid, temporary, fictional
hourly rate); unused hand tool (free, permanent giveaway); surplus equipment (paid, permanent sale,
simulated payment).
BUILD PRIORITY: working end-to-end flow first: discovery, details, quiz, request, review, status
update, handover and return,
timeline. Then waitlist and caps, dashboards, notifications, incidents, polish. Typed data, reusable
components, README with
setup, demo flow, limitations and test checklist. If no real backend is configured, use a clearly
labelled local demo
implementation and do not claim production-grade security.
```

Useful follow-up prompts

- Implement the resource detail page and show every listing field with readable labels and clear safety information.
- Implement requester and owner dashboards on the same request data model so status changes are consistent.
- Add server-side validation so two confirmed bookings cannot overlap for the same resource.
- Add the booking cap and first-eligible waitlist with logged decisions and plain-language reasons.
- Audit the app for role-permission bugs, especially owner-only review actions and access to incident evidence.
- Add the collection and return workflow with before-and-after condition notes and timestamps.
- Add the permanent giveaway and sale handover workflow without reusing temporary booking statuses.
- Add the staged overdue workflow with reminder, flag, grace period and reviewed restriction.
- Run through the acceptance checklist and list anything still mocked, insecure or incomplete.
- Improve mobile layout, loading and empty states, validation messages and accessibility without removing workflow functionality.

32. Alignment with the problem statement and judging criteria

Criterion (points)	Where UNLOCK answers it
Understanding the problem (15)	Personas, hypotheses and evidence plan; risk-based thinking about access, trust, safety and accountability.
Core solution (20)	Full lifecycle from listing to return, useful to both owner and requester.
Working end-to-end product (20)	Demo path: discover, quiz, request, review, approve, check-out, return, timeline.
Technical depth and reliability (15)	Server-side rules, transactional overlap checks, state transitions, audit log, waitlist logic.
Trust, safety, accountability (10)	Eligibility tiers, condition records, incidents treated as allegations, human review, owner spot-checks.
Fair allocation and accessibility (10)	Caps, first-eligible waitlist, published rules, decision reasons, accessibility measures.
Originality and real-world potential (10)	Risk-based approval plus fairness plus accountable handover; designed for many organisations.

Coverage of the expected-strength list: listings with conditions; search and filters; profiles with verification; eligibility checks; approval, rejection and waitlist; conflict handling; check-in and check-out; condition reports; damage, delay and incident reporting; notifications; fair-usage caps; role-based access; auditable history; privacy and security controls; optional reviewable AI.

33. Known limitations

- Payments, verification and email are simulated unless a real service is configured.
- Personas and empathy maps are hypotheses until interviews are done.
- Seeded counts and facilities are fictional and must not be presented as real impact.
- Security is prototype-level, not production-grade.
- The prototype serves one institution; cross-organisation features are designed but not built.
- AI output can be wrong, so it is advisory only and has a fallback.
- Real safety, liability and damage-compensation rules need legal and institutional review before deployment.
- Photo evidence can be unclear or faked, so the human review step cannot be skipped.

34. Future development

- Multi-organisation rollout with shared discovery and per-organisation rules.

- Real identity, institutional login and certificate verification.
- Real payments, deposits and refunds under published terms.
- Email, SMS and calendar integrations.
- Equipment usage logs and sensor data to support condition checks.
- Utilisation analytics and demand forecasting.
- Smarter matching, still reviewable and overridable by people.
- Insurance, damage-fee and appeals processes.
- Multilingual support.

35. Pitch summary

UNLOCK unlocks the value of underused equipment and spaces by making resources easier to discover and access. Requesters compare options and submit clear requests; owners control availability, eligibility and approvals; supervisors document safe handover and return. The key differentiator is not simply listing resources. It is combining discovery with transparent access rules, risk-based approval, fair waitlists and caps, conflict-aware booking, and accountable handover with a complete audit trail.

36. What to validate next

- Interview 3 to 5 potential requesters about how they find equipment, what blocks access, and how they judge price and trust.
- Interview 2 to 3 makerspace owners or supervisors about listing effort, training, safety, scheduling and damage handling.
- Ask which resources can safely be shared, which need supervision, and which must never be booked through an open platform.
- Test whether requesters understand borrow, rent, giveaway and sale.
- Check that the default caps, grace periods, waitlist offer windows, extension rules and data retention periods suit real facilities.
- Use the results to update personas, empathy maps and MVP priorities, and report sample size and limitations honestly.

37. Reference platforms for inspiration

These are references for studying resource discovery or controlled equipment access, not systems UNLOCK depends on.

- I-STEM India (istem.gov.in): a reference for discovering scientific equipment and facilities.
- IIT Bombay Microfactory booking (makerspace.iitb.ac.in/microfactory/booking): a reference for equipment access and training-based permissions.

Before copying any feature, verify the current page and access process. UNLOCK builds its own demo workflow and must not imply integration with these services.

Final implementation principle

For a 48-hour build, a small, reliable end-to-end workflow is more persuasive than many unfinished features. Build the core flow first, label simulated features honestly, enforce the most important safety, fairness and permission rules, and present problem assumptions as hypotheses until interviews support them.